

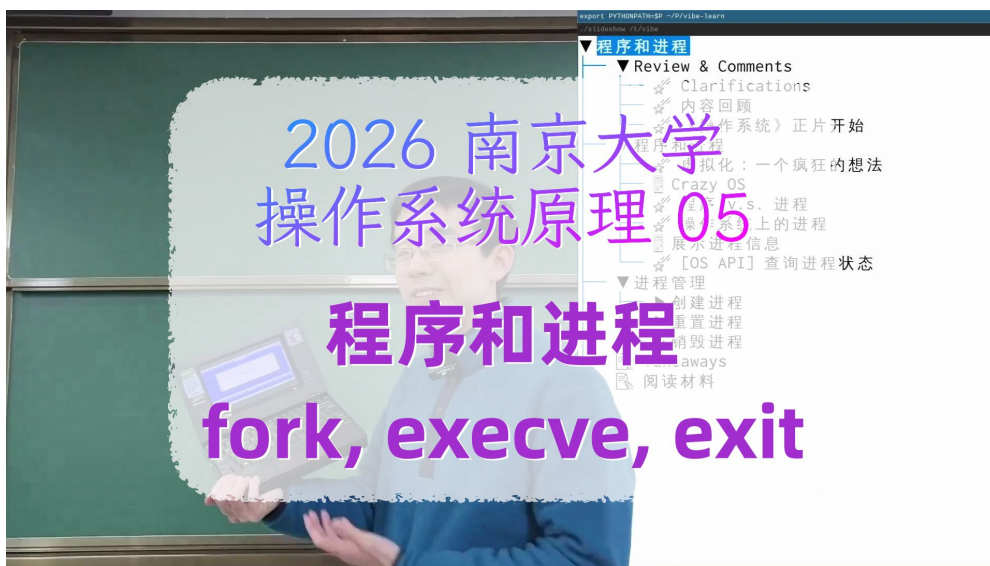
课程笔记

程序和进程；fork, execve, exit

南京大学 操作系统原理 2026 春季学期 第 05 讲

lecture-to-notes

2026 年 3 月 24 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 25 分钟

视频链接：<https://www.bilibili.com/video/BV1N8w9zJENu>

目录

1	Review & Comments	2
1.1	本章小结	2
2	CrazyOS: 一个疯狂的想法	2
2.1	CrazyOS 的构建	3
2.2	CrazyOS 的核心代码	4
2.3	本章小结	5
3	程序与进程	5
3.1	用 GDB 调试 CrazyOS	5
3.2	/proc 文件系统: 进程的“X 光片”	5
3.3	本章小结	6
4	进程管理 API: fork, execve, exit	6
4.1	fork(): 复制当前进程	7
4.2	execve(): 替换当前进程的程序	9
4.3	exit(): 终止当前进程	10
4.4	fork + execve = 创建新进程运行新程序	11
4.5	本章小结	11
5	总结与延伸	11
5.1	讲者的核心信息	11
5.2	结构化提炼	11
5.3	跨章节关联	12
5.4	与前四讲的关联	12
5.5	开放问题	12
5.6	拓展阅读	12

1 Review & Comments

jyy 用开头几分钟回顾了前几讲的要点和同学们的反馈，然后引出本讲的核心主题：**虚拟化**——操作系统如何让一个 CPU “同时” 运行多个程序。

从第 03 讲到第 05 讲的逻辑链

- 第 03 讲：硬件只是无情执行指令的状态机，不知道有没有 OS
- 第 04 讲 (Aside)：AI/Agent 时代操作系统知识仍然重要
- 第 05 讲：操作系统如何在一台物理机上“虚拟化”出多个独立运行的程序——**进程**

1.1 本章小结

本讲从虚拟化的概念出发，通过 CrazyOS 演示、/proc 文件系统探索、再到 fork/execve/exit 三个核心 API，建立了完整的进程管理知识体系。

2 CrazyOS：一个疯狂的想法

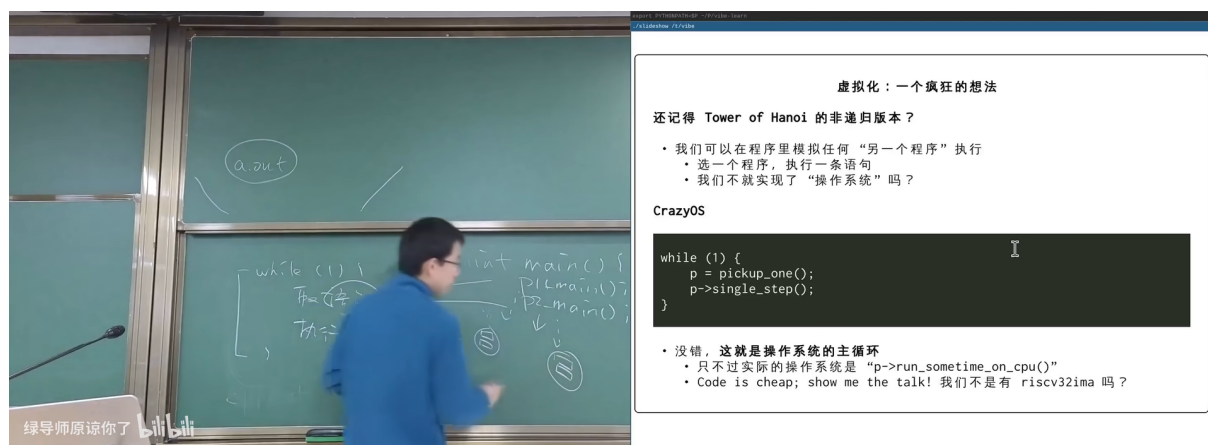


图 1: 虚拟化：还记得 Tower of Hanoi 的非递归版本？CrazyOS 的核心循环¹

jyy 提出了一个“疯狂的想法”——还记得汉诺塔的非递归版本吗？我们可以在程序里模拟“另一个程序”执行：

```
1 while (1) {  
2     p = pick(p);           // 选择一个进程  
3     p->single_step();      // 执行一步  
4 }
```

Listing 1: CrazyOS：操作系统的主循环

¹ 视频画面时间区间：00:08:39–00:09:09。帧实际内容：slide 展示“虚拟化：一个疯狂的想法”，CrazyOS 的 while(1) 主循环代码。

CrazyOS 的核心洞察

这就是操作系统的主循环！

- 只不过实际的操作系统是 “p->run_sometime_on_cpu()”
- Code is cheap; show me the talk! 我们不是有 riscv32ima 吗？

操作系统本质上就是一个不断“选择进程 → 让它跑一会儿”的循环。虚拟化的核心就是**分时复用**——让每个进程以为自己独占了整台机器。

2.1 CrazyOS 的构建

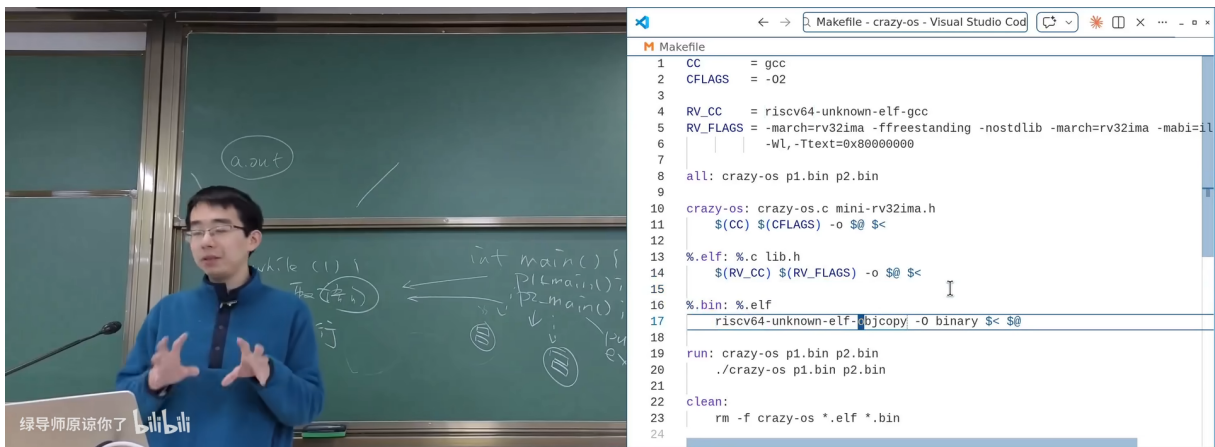


图 2: CrazyOS 的 Makefile: RISC-V 交叉编译配置²

jyy 现场展示了 CrazyOS 的构建系统：

```
1 CC      = gcc
2 CFLAGS  = -O2
3
4 RV_CC    = riscv64-unknown-elf-gcc
5 RV_FLAGS = -march=rv32ima -ffreestanding -nostdlib ...
6
7
8 all: crazy-os p1.bin p2.bin
9
10 crazy-os: crazy-os.c mini-rv32ima.h
11 $(CC) $(CFLAGS) -o $@ $<
```

Listing 2: CrazyOS Makefile

交叉编译的意义

CrazyOS 在 x86/ARM 主机上编译，但运行的“进程”是 RISC-V 32 位程序。这就是交叉编译——在一种架构上编译另一种架构的代码。这也是嵌入式和操作系统开发的日常。

²视频画面时间区间：00:13:39-00:14:09。帧实际内容：VS Code 中打开的 Makefile，显示 CC=gcc, RV_CC=riscv64-unknown-elf-gcc 等交叉编译设置。

2.2 CrazyOS 的核心代码

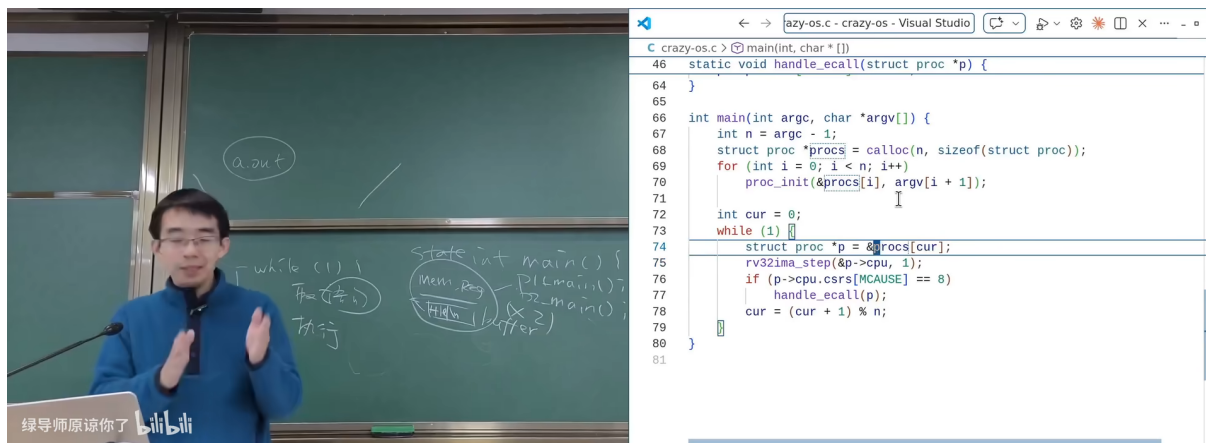


图 3: crazy-os.c: handle_ecall + main 函数的进程调度循环³

```
1 int main(int argc, char *argv[]) {
2     int n = argc - 1;
3     struct proc *procs = calloc(n, sizeof(struct proc));
4     for (int i = 0; i < n; i++)
5         proc_init(&procs[i], argv[i + 1]);
6
7     int cur = 0;
8     while (1) {
9         struct proc *p = &procs[cur];
10        mini_single_step(p->cpu, &p->mem);
11        if (p->cpu.extra[MCAUSE] == 8)
12            handle_ecall(p);
13        cur = (cur + 1) % n;
14    }
15 }
```

Listing 3: CrazyOS 主函数：进程初始化与调度

CrazyOS 实现了什么

- 进程抽象：每个“进程”是一个 struct proc（包含 CPU 状态 + 内存）
- 调度：Round-Robin——依次执行每个进程的一条指令
- 系统调用：通过 ecall 指令陷入，由 handle_ecall 处理
- 整个“操作系统”不到 100 行 C 代码——但它确实能同时运行多个程序！

³视频画面时间区间：00:17:24–00:17:54。帧实际内容：VS Code 中 crazy-os.c 的 main 函数，包含 proc_init、while(1) 循环、handle_ecall 处理。

2.3 本章小结

CrazyOS 用不到 100 行代码演示了操作系统虚拟化的核心：分时复用（Round-Robin 调度）+ 系统调用（ecall 处理）。每个进程以为自己独占 CPU，但实际上它们在轮流执行。

3 程序与进程

从 CrazyOS 的玩具版本到真实 Linux——真实操作系统是如何管理进程的？

3.1 用 GDB 调试 CrazyOS

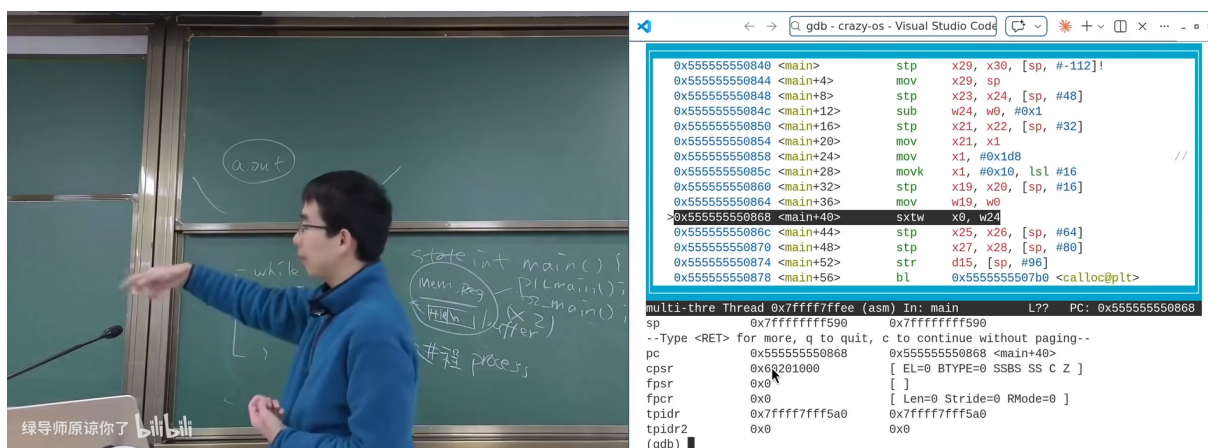


图 4: GDB 调试 crazy-os：查看 RISC-V 汇编级执行⁴

3.2 /proc 文件系统：进程的“X 光片”

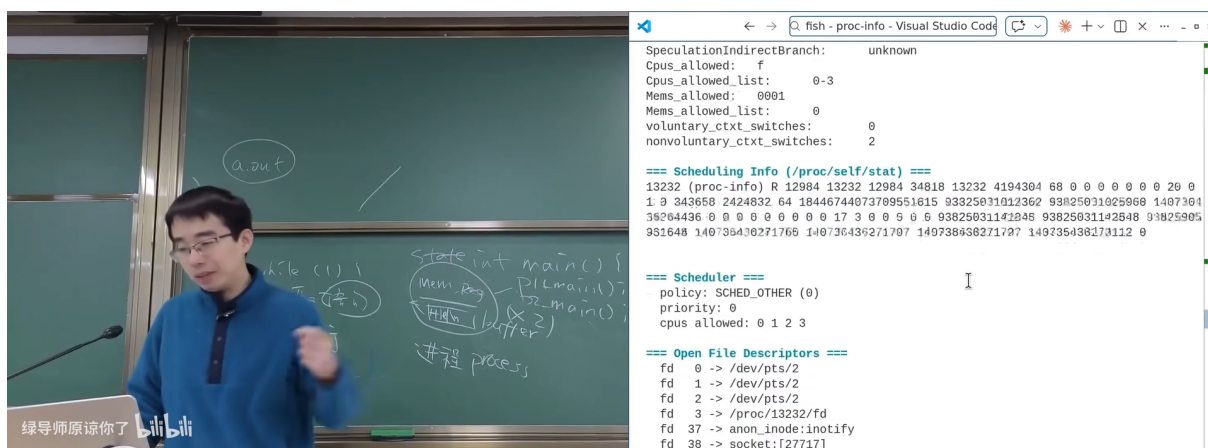


图 5: /proc/self/stat：查看进程的调度信息、CPU 亲和性、文件描述符⁵

jyy 展示了 /proc 文件系统——Linux 暴露进程内部信息的标准接口：

⁴视频画面时间区间：00:22:38–00:23:08。帧实际内容：VS Code + GDB 终端，显示 aarch64 汇编指令（mov, str, ldr, stp 等）和 calloc 调用。

⁵视频画面时间区间：00:27:38–00:28:08。帧实际内容：fish shell 中显示 proc-info 工具输出，包含 Scheduling Info (/proc/self/stat)、policy: SCHED_OTHER、Open File Descriptors (fd 0-38)。

- `/proc/self/stat`: 进程状态 (PID、状态、CPU 时间、内存、上下文切换次数.....)
- `/proc/self/status`: 可读的进程概要
- Scheduling Info: 调度策略 (SCHED_OTHER)、优先级、CPU 亲和性
- Open File Descriptors: 打开的文件描述符列表 (fd 0 = stdin, fd 1 = stdout, fd 38 = socket)

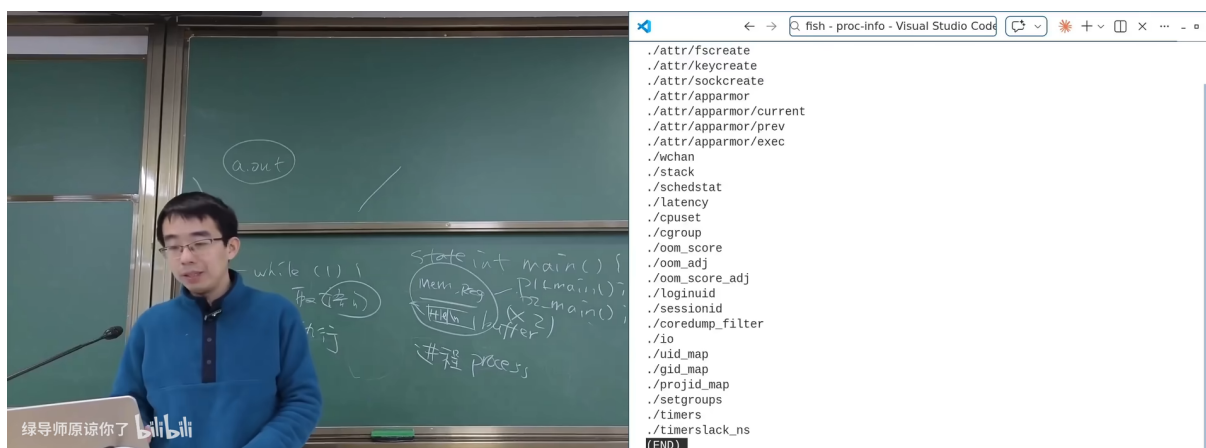


图 6: `/proc/self/attr` 目录: 进程的安全属性列表⁶

`/proc` 是操作系统的“调试接口”

`/proc` 不是真正的文件系统——它是内核动态生成的虚拟文件。每个 `/proc/[pid]/` 目录对应一个进程，里面的“文件”实际上是内核数据结构的文本化表示。这就是 Linux 的哲学：一切皆文件——即使是进程内部状态。

3.3 本章小结

程序是静态的代码 + 数据；进程是程序的运行实例 (= 程序 + 状态)。`/proc` 文件系统让你能像读文件一样读取进程的全部内部状态——PID、调度策略、内存映射、文件描述符、安全属性。

4 进程管理 API: `fork`, `execve`, `exit`

这是本讲的核心——三个改变世界的系统调用。

⁶视频画面时间区间: 00:31:23-00:31:53。帧实际内容: fish shell 中 `ls` 显示 `/proc/self/attr` 目录内容, 包含 `apparmor`、`fscreate`、`keycreate`、`sockcreate` 等安全属性文件。

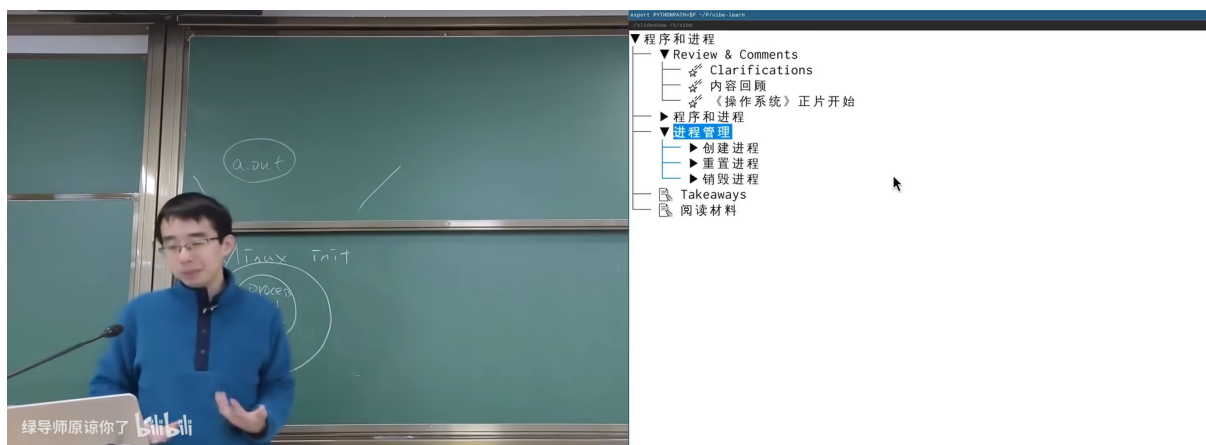


图 7: 课程大纲：本节聚焦“进程管理”⁷

4.1 fork(): 复制当前进程

fork() 的语义

fork() 创建一个当前进程的**完整副本**：

- 子进程获得父进程的内存、寄存器、文件描述符的**拷贝**
- 父进程返回子进程的 PID (> 0)
- 子进程返回 0
- 之后两个进程独立运行，互不影响

$$\text{fork()} = \begin{cases} \text{pid} > 0 & \text{在父进程中} \\ 0 & \text{在子进程中} \\ -1 & \text{失败} \end{cases}$$

⁷ 视频画面时间区间：00:36:00–00:36:30。帧实际内容：课程 Wiki 大纲，“进程管理”项高亮，包含 Review、CrazyOS、程序与进程、进程管理等节。

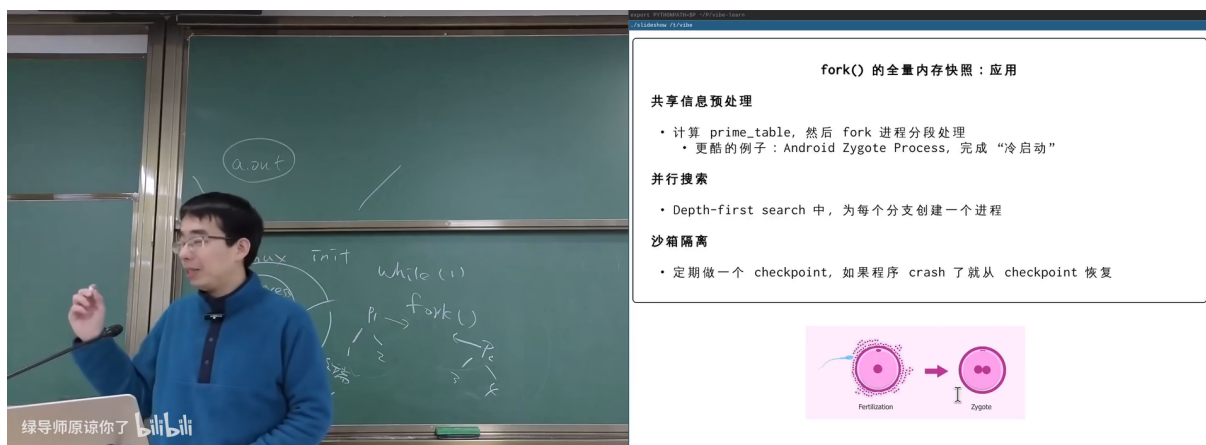


图 8: fork() 的业界内存快照应用：共享计算、Android Zygote、DFS 并行、Checkpoint⁸

fork 的实际应用场景：

共享信息预处理

- 计算 prime_table, 然后 fork——子进程共享计算结果
- **Android Zygote Process**: 预加载所有常用库, fork 出每个 App——“冷启动”变“热启动”

并行搜索

- Depth-first search 中, 为每个分支创建一个进程

沙箱隔离

- 定期做一个 checkpoint (fork), 如果程序 crash 了就从 checkpoint 恢复

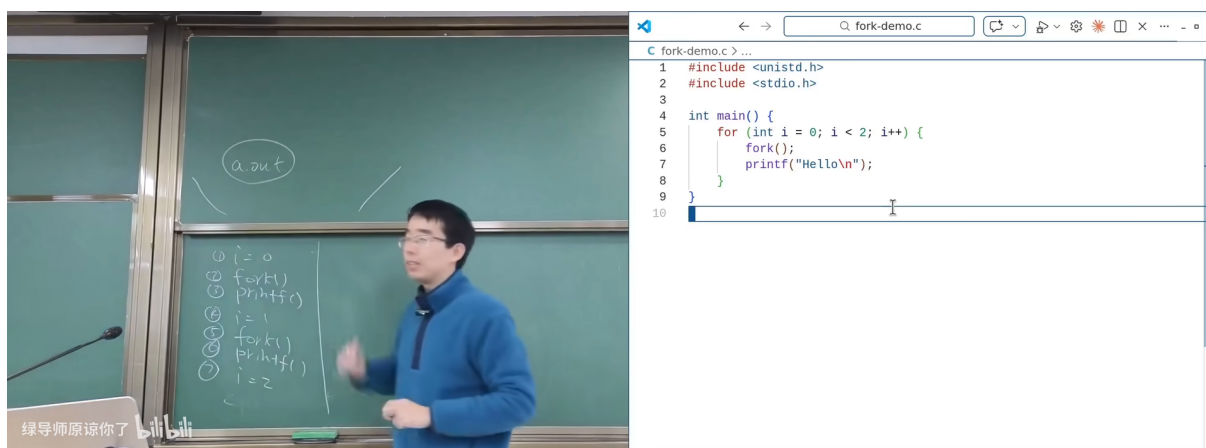


图 9: fork-demo.c: for 循环中 fork + printf 的经典示例⁹

```
1 #include <unistd.h>
2 #include <stdio.h>
```

⁸视频画面时间区间:00:53:29–00:53:59。帧实际内容:slide “fork() 的业界内存快照：应用”，列出共享信息预处理 (prime_table, Android Zygote “冷启动”)、并行搜索 (DFS 分支创建进程)、沙箱隔离 (checkpoint + crash 恢复)。

⁹视频画面时间区间: 01:03:29–01:03:59。帧实际内容: VS Code 中 fork-demo.c 源码, for (int i=0; i<2; i++) { fork(); printf(“Hello”); }。黑板上有 jyy 手绘的进程树。

```

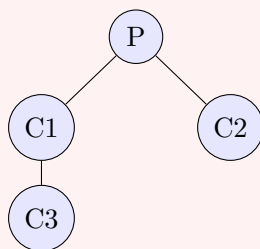
3
4 int main() {
5     for (int i = 0; i < 2; i++) {
6         fork();
7         printf("Hello\n");
8     }
9 }

```

Listing 4: fork-demo.c: 经典的 fork 炸弹雏形

fork 的陷阱：进程数指数增长

上面的代码会打印多少个“Hello”？不是 4 个，而是 **6 个**！



第一次 fork: 1 个进程变 2 个 (各打印 1 次 = 2 次)。第二次 fork: 2 个进程各 fork 一次变 4 个 (各打印 1 次 = 4 次)。总计 2 + 4 = 6 次。如果把 `i < 2` 改成 `i < 30`, 就是经典的 **fork 炸弹**。

4.2 execve(): 替换当前进程的程序

execve() 的语义

`execve(path, argv, envp)` 将当前进程的代码和数据**完全替换**为指定程序：

- 当前进程的代码段、数据段、堆、栈全部被新程序替换
- PID 不变——还是同一个“进程”，但跑的程序完全不同了
- 文件描述符默认保留（除非设置了 `FD_CLOEXEC`）
- **成功时不返回**——因为“你”已经不存在了

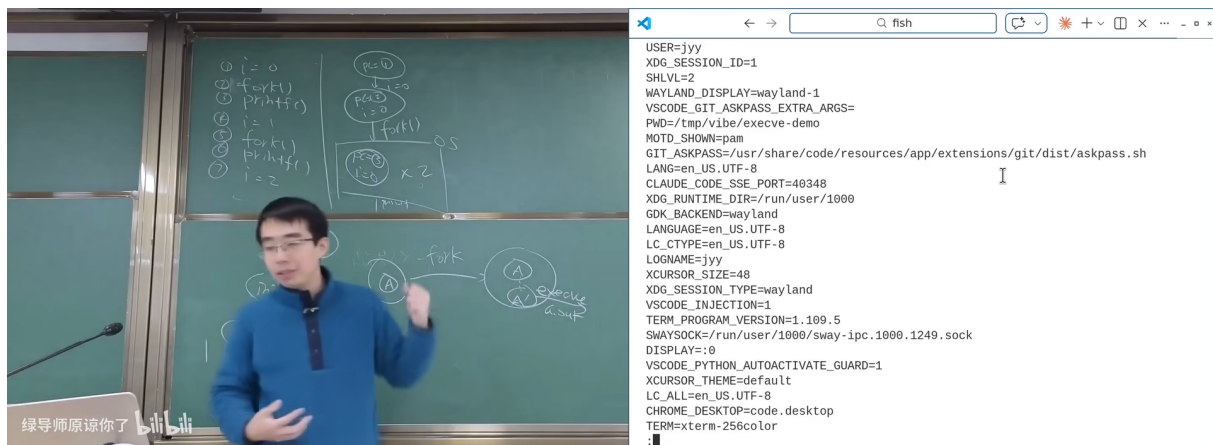


图 10: execve 传递的环境变量: fish shell 中 env 命令的输出¹⁰

环境变量

当你在 shell 中输入命令时, shell 调用 `execve(cmd, argv, environ)`。第三个参数 `environ` 就是环境变量——`PATH`、`HOME`、`USER` 等。有趣的是 `jyy` 的环境中有 `CLAUDE_CODE_SSE_PORT=45848`——他正在用 Claude Code!

4.3 `exit()`: 终止当前进程

`exit()` 的语义

`_exit(status)` 终止当前进程并返回退出码:

- 释放所有资源 (内存、文件描述符)
- 向父进程发送 `SIGCHLD` 信号
- 退出码 0 = 成功, 非零 = 错误
- C 库的 `exit()` 会先执行 `atexit` 注册的清理函数, 再调用 `_exit`

¹⁰视频画面时间区间: 01:18:29-01:18:59。帧实际内容: fish shell 终端显示大量环境变量 (`USER=jyy`, `DISPLAY=:wayland-1`, `CLAUDE_CODE_SSE_PORT=45848`, `VSCODE_INJECTION=1` 等)。

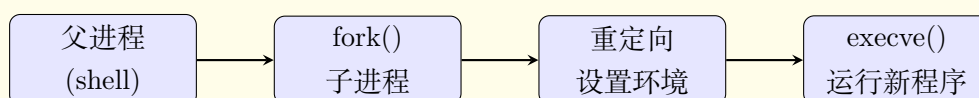
4.4 fork + execve = 创建新进程运行新程序

UNIX 进程创建的设计哲学

为什么不直接提供一个“创建新进程运行新程序”的 API? UNIX 的答案是将其**分解**为两步:

1. `fork()`: 复制当前进程
2. `execve()`: 在子进程中替换为新程序

好处: 在 `fork` 和 `execve` 之间, 你可以做任何操作——重定向文件描述符、设置环境变量、改变工作目录、设置信号处理..... 这种**组合性**是 UNIX 哲学的精髓。



Windows 的 `CreateProcess` 用一个函数做了 `fork+execve` 的所有事——但也因此丧失了中间步骤的灵活性。

4.5 本章小结

三个系统调用构成了 UNIX 进程管理的完整基础: `fork` (复制)、`execve` (替换)、`exit` (终止)。 `fork+execve` 的两步式设计提供了极大的灵活性——这是 UNIX “小工具组合” 哲学在系统调用层面的体现。

5 总结与延伸

5.1 讲者的核心信息

第 05 讲建立了从“虚拟化概念”到“具体 API”的完整链条:

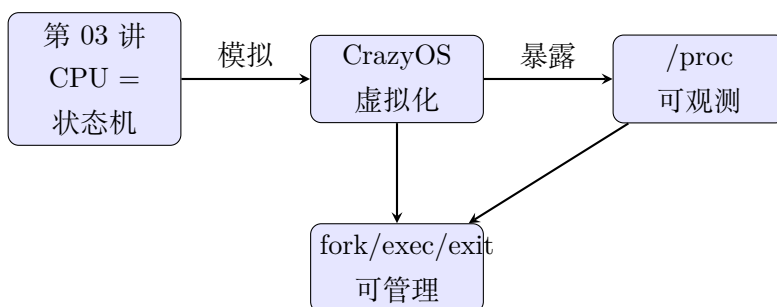
虚拟化想法 $\xrightarrow{\text{CrazyOS}}$ 进程抽象 $\xrightarrow{\text{/proc}}$ 进程可观测 $\xrightarrow{\text{fork/execve/exit}}$ 进程可管理

5.2 结构化提炼

四个核心 Takeaway

1. **CrazyOS 证明了 OS 的简单性**——不到 100 行 C 代码就能实现进程虚拟化。核心循环: `pick` $\rightarrow$ `run` $\rightarrow$ `handle_ecall` $\rightarrow$ `pick`。
2. **/proc 让进程透明**——进程的全部状态 (PID、调度、内存、文件、安全属性) 都可以通过读文件获取。“一切皆文件”的哲学在这里得到完美体现。
3. **fork 是“复制”而非“创建”**——子进程是父进程的完整快照。这个看似奇怪的设计带来了 Android Zygote、checkpoint/restore 等强大应用。
4. **fork + execve 的分离是设计智慧**——在两步之间插入任意操作 (重定向、环境设置) 的能力, 是 UNIX shell 强大的根本原因。

5.3 跨章节关联



5.4 与前四讲的关联

- 第 02 讲的 `strace` 追踪 `execve` → 第 05 讲解释了 `execve` 的精确语义
- 第 02 讲说 “gcc 调用了 14 个 `execve`” → 第 05 讲解释了为什么要 `fork+execve` 而非直接创建
- 第 03 讲说 “CPU Reset 后 Firmware 加载 OS” → 第 05 讲说 “OS 用 `fork+execve` 启动所有用户程序”

5.5 开放问题

1. `fork` 的 Copy-on-Write 具体是怎么实现的? → 后续的 “虚拟内存” 章节
2. 当 `fork` 出的子进程很多时, OS 如何高效调度? → 后续的 “调度” 章节
3. 容器 (Docker) 是如何在 `fork/execve` 基础上实现进程隔离的?

5.6 拓展阅读

- jyy 的课程 Wiki: <https://jyywiki.cn/>
- OSTEP Chapter 5: Process API
- `man 2 fork`, `man 2 execve`, `man 2 exit`